Ramakrishna Mission Vidyamandira

(An Autonomous College Under University of Calcutta)

Computer Science (Honors) Semester VI 2026

Paper: 6CMSMJC2 Practical

Submitted by
Class Roll Number: 340 B.Sc. 6th Semester Batch: 2023-27

Q1)

1. The following table represents the data set along with corresponding classes.

BRIGHTNESS	SATURATION	CLASS
40	20	Red
50	50	Blue
60	90	Blue
10	25	Red
70	70	Blue
60	10	Red
25	80	Blue

Find out the class of new data entry as follows:

BRIGHTNESS	SATURATION	CLASS
20	35	?

CODE:

```
import math

# i have solved this problem using K-nearest neighbors algorithm.

# dataset (brightness, saturation, color)
data = [
    (40, 20, "Red"),
    (50, 50, "Blue"),
    (60, 90, "Blue"),
    (10, 25, "Red"),
    (70, 70, "Blue"),
    (60, 10, "Red"),
    (25, 80, "Blue")
]

# test point (brightness, saturation)
test = (20, 35)

# calculate distance
distances = []

for point in data:
    x, y, label = point
    d = math.sqrt((x - test[0])**2 + (y - test[1])**2)
    distances.append((d, label))

# sort distances
distances.sort()

# take K=3
```

```python
k = 3
neighbors = distances[:k]

count = {"Red":0, "Blue":0}

for d, label in neighbors:
    count[label] += 1

# prediction
if count["Red"] > count["Blue"]:
    print("Class = Red")
else:
    print("Class = Blue")
```

OUTPUT:

Class = Red

Q2)

2. The following table represents the survey results from the 7 online stores:

On-line store	Monthly E-commerce sales (unit)	Online Advertising Dollars
1	368	1.7
2	340	1.5
3	665	2.8
4	954	5
5	331	1.3
6	556	2.2
7	376	1.3

Find out the equation of the best-fit straight line.

CODE:

```
# i have solved this problem using linear regression.

# this is the dataset of the survey results from the 7 online stores
x = [1.7, 1.5, 2.8, 5.0, 1.3, 2.2, 1.3]
y = [368, 340, 665, 954, 331, 556, 376]

n = len(x)

sum_x = sum(x)
sum_y = sum(y)

sum_x2 = sum([i*i for i in x])
sum_xy = sum([x[i]*y[i] for i in range(n)])

# slope (m)
m = (n*sum_xy - sum_x*sum_y) / (n*sum_x2 - sum_x**2)

# intercept (c)
c = (sum_y - m*sum_x) / n

print("Slope (m):", m)
print("Intercept (c):", c)

print(f"Equation: y = {m:.2f}x + {c:.2f}")
```

OUTPUT:

Slope (m): 171.4655639913232
Intercept (c): 125.83486984815623
Equation: y = 171.47x + 125.83

Q3)

3. A company manufactures an electronic device to be used in a very wide temperature range. The company knows that increased temperature shortens the life time of the device, and a study is therefore performed in which the life time is determined as a function of temperature. The following data is found:

Temperature in Celcius (t)	10	20	30	40	50	60	70	80	90
Life time in hours (y)	420	365	285	220	176	117	69	34	5

a) Calculate the value of Life time in hour for 95% confidence interval of Temperature in the usual linear regression model.

b) Calculate the 95% confidence interval for the slope in the usual linear regression model, which expresses the life time as a linear function of the temperature.

CODE:

```
import math

# i have solved this problem using linear regression.

# Dataset (temparature, life time)
t = [10,20,30,40,50,60,70,80,90]
y = [420,365,285,220,176,117,69,34,5]

n = len(t)

sum_t = 0
sum_y = 0
sum_t2 = 0
sum_ty = 0

print("t   y   t^2   t*y")

for i in range(n):
    ti = t[i]
    yi = y[i]

    t2 = ti * ti
    ty = ti * yi

    print(ti, yi, t2, ty)
```

```
    sum_t += ti
    sum_y += yi
    sum_t2 += t2
    sum_ty += ty

print("\nSummations:")
print("Sum_t =", sum_t)
print("Sum_y =", sum_y)
print("Sum_t2 =", sum_t2)
print("Sum_ty =", sum_ty)

# Means
mean_t = sum_t / n
mean_y = sum_y / n

print("\nMean t =", mean_t)
print("Mean y =", mean_y)

# Regression Model
b = (n*sum_ty - sum_t*sum_y) / (n*sum_t2 - sum_t**2)
a = (sum_y - b*sum_t) / n

print("\nRegression Equation:")
print("y =", a, "+", b, "* t")

# Sxx
Sxx = 0
for ti in t:
    Sxx += (ti - mean_t)**2

# SSE
SSE = 0
for i in range(n):
    y_hat = a + b*t[i]
    SSE += (y[i] - y_hat)**2

# Standard error
s = math.sqrt(SSE / (n - 2))

# t value for 95% CI (df = 7)
t_val = 2.365

print("\n95% CI for Life Time......")

# take mean temperature (t = 50)
t0 = mean_t

y_hat = a + b*t0

margin = t_val * s * math.sqrt(1/n)

lower = y_hat - margin
```

```
upper = y_hat + margin

print("Predicted Life Time =", y_hat)
print("Confidence Interval =", lower, "to", upper)

print("\n95% CI for Slope......")

SE_b = s / math.sqrt(Sxx)

lower_b = b - t_val * SE_b
upper_b = b + t_val * SE_b

print("Slope (b) =", b)
print("Confidence Interval =", lower_b, "to", upper_b)
```

OUTPUT:

```
t  y  t^2  t*y
10 420 100 4200
20 365 400 7300
30 285 900 8550
40 220 1600 8800
50 176 2500 8800
60 117 3600 7020
70 69 4900 4830
80 34 6400 2720
90 5 8100 450

Summations:
Sum_t = 450
Sum_y = 1691
Sum_t2 = 28500
Sum_ty = 52670

Mean t = 50.0
Mean y = 187.88888888888889

Regression Equation:
y = 453.55555555555554 + -5.3133333333333335 * t

95% CI for Life Time......
Predicted Life Time = 187.88888888888886
Confidence Interval = 172.26981835738547 to 203.50795942039224

95% CI for Slope......
Slope (b) = -5.3133333333333335
Confidence Interval = -5.91825733385085 to -4.708409332815817
```

Q4)

4. Evaluate the following data-set to fit a multiple linear regression model.

Y	X1	X2
140	60	22
155	62	25
159	67	24
179	70	20
192	71	15
200	72	14
212	75	14
215	78	11

CODE:

```
import numpy as np

# Data
Y = np.array([140,155,159,179,192,200,212,215])

X1 = np.array([60,62,67,70,71,72,75,78])
X2 = np.array([22,25,24,20,15,14,14,11])

# Create  Matrix
# Add intercept (1)
X = []

for i in range(len(X1)):
    X.append([1, X1[i], X2[i]])

X = np.array(X)

print("Design Matrix X:\n", X)

# apply formula B = (X^T * X)^-1 * X^T * Y
# transpose
XT = X.T

# multiply
XT_X = XT.dot(X)

# inverse
XT_X_inv = np.linalg.inv(XT_X)

# final coefficients
B = XT_X_inv.dot(XT).dot(Y)

print("\nCoefficients:", B)
```

```
# Final Equation
b0 = B[0]
b1 = B[1]
b2 = B[2]

print("\nFinal Equation:")
print("Y =", b0, "+", b1, "* X1 +", b2, "* X2")
```

OUTPUT:

Design Matrix X:
[[1 60 22]
[1 62 25]
[1 67 24]
[1 70 20]
[1 71 15]
[1 72 14]
[1 75 14]
[1 78 11]]

Coefficients: [-6.86748725 3.1478931 -1.65614327]

Final Equation:
Y = -6.8674872477282065 + 3.147893102683577 * X1 + -1.656143269017738 * X2

Q5)

5. The data-set of pass and fail in an exam of 5 students given in the following table:

Study_in_Hours	Pass(1)_Fail(0)
29	0
15	0
33	1
28	1
39	1

Use logistic regression to answer the following questions:

1. Calculate the probability of pass for the students who studied 33 hours.
2. At least how many hours student should study that makes he will pass the course with the probability of more than 95%.

The following model is suggested for odds of passing the course is:

log(odds) = -64 + 2*hours

CODE:

```
import math

# DATASET(study hours, pass/fail)
hours_data = [29, 15, 33, 28, 39]
result_data = [0, 0, 1, 1, 1]

print("Given Dataset")
print("Hours\tResult (Pass=1, Fail=0)")

for i in range(len(hours_data)):
    print(hours_data[i], "\t", result_data[i])

# GIVEN MODEL
# log(odds) = -64 + 2 * hours
# Probability for 33 hours
hours = 33

# Calculate log(odds)
log_odds = -64 + 2 * hours

#Convert to probability
probability = 1 / (1 + math.exp(-log_odds))

print("\nPart 1")
print("Hours =", hours)
print("Log(odds) =", log_odds)
print("Probability of passing =", probability)

#Minimum hours for >95% probability
p_target = 0.95
```

```python
# Convert to log(odds)
log_odds_target = math.log(p_target / (1 - p_target))

# Solve equation
required_hours = (log_odds_target + 64) / 2

print("\nPart 2")
print("Target Probability =", p_target)
print("Log(odds) target =", log_odds_target)
print("Minimum hours required =", required_hours)

# FINAL RESULT

print("\nFinal Answer")
print("Probability at 33 hours ≈", round(probability, 3))
print("Minimum hours required ≈", round(required_hours))
```

OUTPUT:

```
Given Dataset
Hours   Result (Pass=1, Fail=0)
29       0
15       0
33       1
28       1
39       1

Part 1
Hours = 33
Log(odds) = 2
Probability of passing = 0.8807970779778823

Part 2
Target Probability = 0.95
Log(odds) target = 2.9444389791664394
Minimum hours required = 33.47221948958322

Final Answer
Probability at 33 hours ≈ 0.881
Minimum hours required ≈ 33
```